VARIÁVEIS DE AMBIENTE

Declarando uma variável

- minhavar=”minha variável”

#Variável normal (temporária) - só existe no seu script

- $PATH
- $HOME
- $USER

#Variável de ambiente (permanente) - existe no seu sistema

```
[ricardo@nixos-demo:~/scripts]$ minhavar="minha variável"

[ricardo@nixos-demo:~/scripts]$ echo $minhavar
minha variável
```

VARIÁVEIS DE AMBIENTE

$PATH é a mais importante! ➔ É onde o bash procura comandos!

echo $PATH output ➔ /usr/bin:/bin:/usr/local/bin

Quando você digita “ls”(ou qualquer comando) ele procura pelo comando nesses diretórios

VARIÁVEIS DE AMBIENTE

➔ Você pode procurar o caminho de qualquer comando e sua natureza usando type

```
[ricardo@nixos-demo:~]$ type vim
vim is /run/current-system/sw/bin/vim
```

STDIN, STDOUT, STDERR E REDIRECIONAMENTOS

- “echo $?”

$? Guarda o resultado do último processo >

0 = sucesso

e

1 = erro

- stdin 0
- stdout 1
- stderr 2

REDIRECIONAMENTOS

Redirecionamentos:

A diferença entre > e >>:

comando > arquivo.txt # cria/SOBRESCREVE o arquivo

comando >> arquivo.txt # cria/ADICIONA ao final

REDIRECIONAMENTOS

Aplicando ao stderr:

comando 2> arquivo.txt # redireciona erro → SOBRESCREVE
comando 2>> arquivo.txt # redireciona erro → ADICIONA ao final

PIPES

comando 1 | comando 2

Quando usamos o operador “|” a saída do primeiro comando deixa de ir direto para a tela e serve como entrada para o segundo comando. Essa lógica permite construir operações mais sofisticadas sem precisar de uma ferramenta única que resolva tudo.

Exemplo:

```
[ricardo@nixos-demo:~]$ find . -name "*hello*"
./eclipse-workspace/.metadata/.plugins/org.eclipse.ui.cheatsheets/org.eclipse.jdt.helloworld.xml
./Cppprojetos/build/Debug/hello.o
./scripts/hello
./.cache/vscode-cpptools/ipch/e3ddce22c0ee34f8/hello.ipch
./.vscode/extensions/vadimcn.vscode-lldb-1.11.5/lldb/lib/python3.12/__phello__
./.vscode/extensions/vadimcn.vscode-lldb-1.11.5/lldb/lib/python3.12/__hello__.py

[ricardo@nixos-demo:~]$ find . -name "*hello*" | grep "vscode"
./.cache/vscode-cpptools/ipch/e3ddce22c0ee34f8/hello.ipch
./.vscode/extensions/vadimcn.vscode-lldb-1.11.5/lldb/lib/python3.12/__phello__
./.vscode/extensions/vadimcn.vscode-lldb-1.11.5/lldb/lib/python3.12/__hello__.py

[ricardo@nixos-demo:~]$
```

O comando find busca por arquivos e diretórios com “hello” no nome a partir do diretório atual, e passa sua saída como entrada para o comando grep, que filtra os resultados exibindo apenas os caminhos que contêm o padrão "vscode".

ANATOMIA DE UM SCRIPT

```
#!/usr/bin/env bash



nome=$1

echo hello $nome
```

A primeira linha é o “shebang#!” - diz qual interpretador vai usar (bash)

variável nome = primeiro argumento

Saída para o usuário: hello [nome]

```
[ricardo@nixos-demo:~/scripts]$ ./hello ricardo
hello ricardo

[ricardo@nixos-demo:~/scripts]$
```

[argumento $1]

Para rodar o script: ./[nome do arquivo]

IF ELSE

```
#!/usr/bin/env bash

read -p "Digite seu nome: " nome

if [[ -z $nome ]]; then
        echo "ERRO! preencha o campo:"

elif [[ $nome == "Ricardo" ]]; then
        echo "Nossa, Temos o mesmo nome!!"

else
        echo "Não temos o mesmo nome!!"



fi
```

Input para o usuário(read -p) e guarda na variável [nome]

if [[ ]]; then

<comando>

fi

Para rodar o script: ./[nome do arquivo]

[[-z]] - checa se a variavel nome está vazia

```bash
#!/usr/bin/env bash


read -p "Qual seu nome: " nome

if [[ -n $nome ]]; then
        ./hello "$nome"
```

Outro script

```bash
else
        echo "ME CONTA SEU NOME!!"

fi
```

FOR

```
#!/usr/bin/env bash

read -p "me diga o nome de várias pessoas: " nomes

for nome in $nomes; do
        ./hello "$nome"

done
```

script hello

for [variável] in [var do input]; do

<comando>

done

```
[ricardo@nixos-demo:~/scripts]$ ./loopfor
me diga o nome de várias pessoas: jess calderon ricardo garcia caramez joao 67 caio
hello jess
hello calderon
hello ricardo
hello garcia
hello caramez
hello joao
hello 67
hello caio
```

prompt e output do programa:

AUTOMAÇÃO

```bash
#!/usr/bin/env bash

read -p "Entre um caminho: " caminho

if [[ -z $caminho ]]; then
        echo "Caminho inválido!"

else
        ls $caminho | sort > ./tmp
        cat ./tmp
fi
```

AUTOMAÇÃO

```
[ricardo@nixos-demo:~/scripts]$ ./conectando
Entre um caminho: /etc
adjtime
alsa
bash_logout
bashrc
binfmt.d
cups
dbus-1
default
firefox
fonts
fstab
fuse.conf
group
host.conf
hostname
hosts
inputrc
ipsec.d
ipsec.secrets
issue
kbd
kernel
libblockdev
locale.conf
localtime
login.defs
lsb-release
```

find - como usar o find

find [caminho] [critério] [ação]

Caminho(O diretório de começo)

```
find .                  #diretório atual
find /home/user         #pesquisa em /home/user
find /                  #pesquisa em todo lugar(lento!)
```

Critério(O que você está procurando)

```
find . -name "*.txt"    #por nome
find . -type f          #por tipo
find . -size +10M       #por tamanho
find . -mtime           #por tempo
```

Ação(O que fazer com os resultados)

```
find . -name "*.log"
#padrão: apenas lista os resultados
find . -name "documento(1).pdf" -delete
# deleta a(s) pasta(s) referenciada(s)
find . -name "*.txt" -exec cat {} \;
# executa o comando cat em cada arquivo encontrado
```

grep

grep [flags] [padrão] [arquivo]

padrão(padrão sendo procurado)

grep “error” system.log #pesquisa pela palavra “error” na pasta “system.log”
grep “Hello” file.txt #pesquisa pela palavra “Hello” na pasta “file.txt”

pasta(onde será procurado)

grep “error” system.log #pesquisa em “system.log”
grep “Hello” arquivo.txt #pesquisa em “file.txt”

Flags (Opcional)

grep -i “error” system.log #permite que a pesquisa não diferencie letras maiúsculas das minúsculas
grep -r “error” /var/log/* #permite a pesquisa pela palavra error em todos os arquivos no dir /log

cut - como usar o cut

2 Modos:

1 - cut -d “Separador” -f [Coluna] [arquivo]

ou

2 - cut [flag] [Posição] [arquivo]

modo 1:

```
[ricardo@nixos-demo:~/scripts]$ cat pessoas.txt
ricardo:21:Brasil
jess:21:Inglaterra
garcia:30:Brasil
caramez:21:Brasil

[ricardo@nixos-demo:~/scripts]$ cut -d ":" -f 1 pessoas.txt
ricardo
jess
garcia
caramez

[ricardo@nixos-demo:~/scripts]$ cut -d ":" -f 1-2 pessoas.txt
ricardo:21
jess:21
garcia:30
caramez:21

[ricardo@nixos-demo:~/scripts]$ cut -d ":" -f 1-3 pessoas.txt
ricardo:21:Brasil
jess:21:Inglaterra
garcia:30:Brasil
caramez:21:Brasil
```

modo 2:

```
[ricardo@nixos-demo:~/scripts]$ cut -c 1-4 pessoas.txt
rica
jess
garc
cara

[ricardo@nixos-demo:~/scripts]$ cut -c 1-12 pessoas.txt
ricardo:21:B
jess:21:Ingl
garcia:30:Br
caramez:21:B

[ricardo@nixos-demo:~/scripts]$ cut -c 1 pessoas.txt
r
j
g
c
```

Outros comandos importantes

wc #conta coisas

```
[ricardo@nixos-demo:~/MyFlake]$ wc flake.lock
 27  46 567 flake.lock
```

Linhas letras bytes

file #descobre o conteúdo de um arquivo

```
[ricardo@nixos-demo:~/scripts]$ file backup
backup: Bourne-Again shell script, ASCII text executable
```

sort #ordena alfabéticamente

less #visualisador de arquivos

head #primeiras 10 linhas de um arquivo

tail #últimas 10 linhas de um arquivo

uniq #remove linhas duplicadas consectutivas

sort #ordena alfabéticamente

```
[ricardo@nixos-demo:~/scripts]$ cat texto.txt
banana
banana
uva
uva
banana
uva
banana
maça
maça
laranja
maça
laranja

[ricardo@nixos-demo:~/scripts]$ uniq texto.txt
banana
uva
banana
uva
banana
maça
laranja
maça
laranja
```

sort + uniq

```
[ricardo@nixos-demo:~/scripts]$ sort texto.txt
banana
banana
banana
banana
laranja
laranja
maça
maça
maça
uva
uva
uva

[ricardo@nixos-demo:~/scripts]$ sort texto.txt | uniq
banana
laranja
maça
uva
```